Chapitre 09 — Redirections et tubes

1. Les trois flux de données

Chaque commande Linux travaille avec trois canaux de communication :



Flux	Numéro	Défaut	Description
`stdin`	0	Clavier	Ce que reçoit la commande
`stdout`	1	Écran	Le résultat normal
`stderr`	2	Écran	Les messages d'erreur

Analogie : imagine une machine à café. Tu mets des grains (stdin), tu obtiens du café (stdout), et si quelque chose cloche la machine affiche une erreur (stderr). Les redirections te permettent de changer d'où viennent les grains et où va le café.

2. Rediriger la sortie vers un fichier

`>` — écrire dans un fichier (écrase)

```
ls /etc > liste.txt         # sauvegarde le résultat dans liste.txt
date > date_du_jour.txt     # écrit la date dans un fichier
```

`>` **écrase** le fichier s'il existe déjà. Son contenu précédent est perdu.

`>>` — ajouter à la fin d'un fichier

```
echo "Ligne 1" > journal.txt    # crée le fichier avec "Ligne 1"
echo "Ligne 2" >> journal.txt   # ajoute "Ligne 2" à la fin
echo "Ligne 3" >> journal.txt   # ajoute "Ligne 3" à la fin
cat journal.txt
# Ligne 1
# Ligne 2
# Ligne 3
```

`2>` — rediriger les erreurs

```
ls /dossier-inexistant 2> erreurs.txt   # redirige l'erreur dans un fichier
ls /etc /dossier-inexistant &> tout.txt # redirige résultat ET erreurs
```

Jeter les erreurs — `/dev/null`

```
ls /dossier-inexistant 2> /dev/null    # l'erreur disparaît silencieusement
```

`/dev/null` est le "trou noir" de Linux — tout ce qu'on y envoie est supprimé.

---

3. Les tubes (pipes) — `|`

Le tube `|` connecte la **sortie d'une commande** à l'**entrée de la suivante**. C'est l'outil le plus puissant du terminal Linux.

> **Analogie** : un tube de plomberie. L'eau sort d'un robinet (commande 1) et entre directement dans le seau (commande 2) sans toucher le sol.

```
commande1 | commande2 | commande3
```

Exemples

```
# Compter les fichiers dans /etc
ls /etc | wc -l

# Chercher un mot dans la liste des processus
ps aux | grep firefox

# Voir les 5 premières lignes de la liste des paquets installés
apt list --installed | head -5

# Chercher un utilisateur dans /etc/passwd
cat /etc/passwd | grep stagiaire

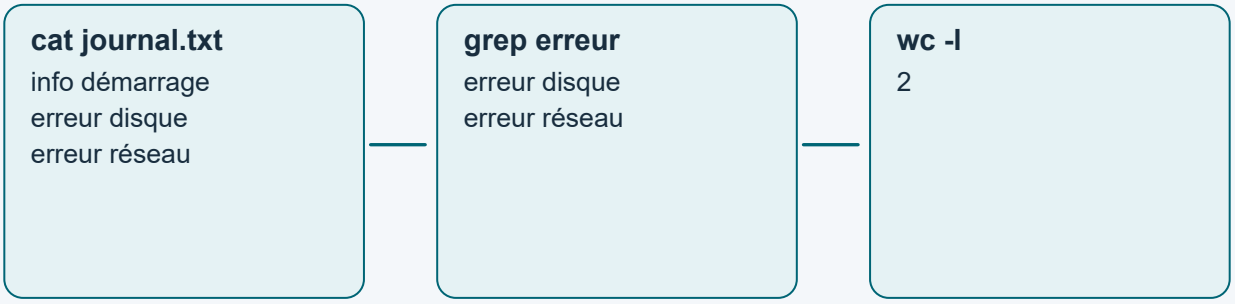
# Combiner tri et comptage
ls /etc | sort | uniq | wc -l
```

illustration-09

09

Suivre les données dans un tube

Chaque commande reçoit le résultat de la commande précédente.




```
cat journal.txt | grep erreur | wc -l
```

3 lignes → 2 lignes sélectionnées → résultat : 2

> remplace un fichier ; >> ajoute à la fin.

Le tube | transmet la sortie normale (stdout). Les erreurs (stderr) ne sont pas incluses par défaut.

Observer les lignes qui passent d'une commande à la suivante.

Observer les lignes qui passent d'une commande à la suivante.

---

4. Rechercher du texte — `grep`

`grep` filtre les lignes qui contiennent un motif :

```
grep "erreur" /var/log/syslog          # lignes contenant "erreur"
grep -i "erreur" /var/log/syslog       # insensible à la casse
grep -n "root" /etc/passwd             # avec numéro de ligne
grep -v "commentaire" fichier.conf     # lignes qui NE contiennent PAS le motif
grep -r "mot" /etc/                    # recherche récursive dans un dossier
```

`grep` combiné avec un tube

```
# Chercher les erreurs dans les logs
cat /var/log/syslog | grep -i "error"

# Compter les erreurs
grep -i "error" /var/log/syslog | wc -l

# Trouver les utilisateurs avec bash
cat /etc/passwd | grep "/bin/bash"
```

---

5. Quelques commandes de traitement utiles

Commande	Rôle
`wc -l`	Compte le nombre de lignes
`sort`	Trie les lignes par ordre alphabétique
`sort -n`	Trie numériquement
`uniq`	Supprime les lignes en double (après un `sort`)
`head -n 5`	Affiche les 5 premières lignes
`tail -n 5`	Affiche les 5 dernières lignes
`cut -d: -f1`	Coupe selon un séparateur et garde le 1er champ

```
# Afficher uniquement les noms d'utilisateurs depuis /etc/passwd
cut -d: -f1 /etc/passwd | sort

# Trouver les 3 plus gros fichiers dans /var/log
du -sh /var/log/* 2>/dev/null | sort -rh | head -3
```

Récapitulatif des commandes

Commande / Symbole	Description
`commande > fichier`	Redirige stdout vers un fichier (écrase)
`commande >> fichier`	Ajoute stdout à la fin d'un fichier
`commande 2> fichier`	Redirige stderr vers un fichier
`commande &> fichier`	Redirige stdout et stderr vers un fichier
`commande 2> /dev/null`	Jette les erreurs
`cmd1 | cmd2`	Connecte la sortie de cmd1 à l'entrée de cmd2
`grep "motif" fichier`	Filtre les lignes contenant le motif
`grep -i`	Insensible à la casse
`grep -v`	Lignes qui ne contiennent PAS le motif
`wc -l`	Compte les lignes
`sort`	Trie les lignes
`head -n N`	Affiche les N premières lignes
`tail -n N`	Affiche les N dernières lignes
`cut -d: -f1`	Découpe une ligne selon un séparateur

À retenir

- Chaque commande a trois flux : `stdin` (entrée), `stdout` (résultat), `stderr` (erreurs).
- `>` redirige stdout vers un fichier en **écrasant** ; `>>` **ajoute** à la fin.
- `2>` redirige uniquement les erreurs ; `&>` redirige les deux.
- `/dev/null` est le trou noir du système — tout ce qu'on y envoie est supprimé.
- Le **tube** `|` connecte la sortie d'une commande à l'entrée de la suivante.
- `grep "motif"` filtre les lignes contenant un texte — indispensable combiné aux tubes.
- `wc -l` compte les lignes, `sort` trie, `head`/`tail` extraient le début/la fin.

TP associé → `tp-09-redirections.md`